

Laboratório
Concorrente

de

Programação

Lab7 - 26.1

Producer Conditional
Consumer Problem

Objetivo

Muitos problemas em Ciência da Computação são resolvidos com uma estratégia que envolve dois grupos de processos: produtores e consumidores. O consumidor, para realizar o seu trabalho, precisa de itens produzidos por processos produtores. Tipicamente, um buffer compartilhado mantém os itens produzidos à espera de consumidores.

Neste laboratório, nós disponibilizamos o código serial de um sistema Produtor-Consumidor que considera um **buffer ilimitado** e **nenhum controle de acesso concorrente**. Vocês devem implementar a concorrência neste sistema, considerando **Produtores Genéricos e Consumidores Condicionais**. Este problema é uma variação do padrão Produtor-Consumidor clássico, em que os consumidores não processam qualquer item do buffer, mas apenas aqueles que satisfazem uma determinada condição.

A sua solução deve lidar com a execução simultânea de produtores e consumidores e o controle de acesso concorrente utilizando **semáforos**. Desenvolva uma solução segura para evitar condições de corrida e outros problemas de sincronização.

Sua implementação deverá garantir os seguintes comportamentos:

1) Buffer limitado e seguro:

- a) O buffer deve ter **capacidade máxima de 100 itens**.

2) Produtores:

- a) Inserem números inteiros aleatórios no buffer.
- b) Devem aguardar se o buffer estiver cheio, evitando sobrescrever dados ainda não consumidos.
- c) Podem operar de forma concorrente entre si.

3) Consumidores:

- a) Existem dois tipos distintos:
 - i) Um que consome apenas números pares (PairConsumer).
 - ii) Outro que consome apenas números ímpares (OddConsumer).
- b) Os consumidores devem remover itens do buffer de forma concorrente, sem acessar posições inválidas ou consumir o mesmo item.
- c) Se um consumidor remover um item que não atende à sua condição, ele deve reinserir o item no buffer.

d) Caso o buffer esteja vazio, o consumidor deve aguardar até que novos itens sejam produzidos.

4) Concorrência robusta:

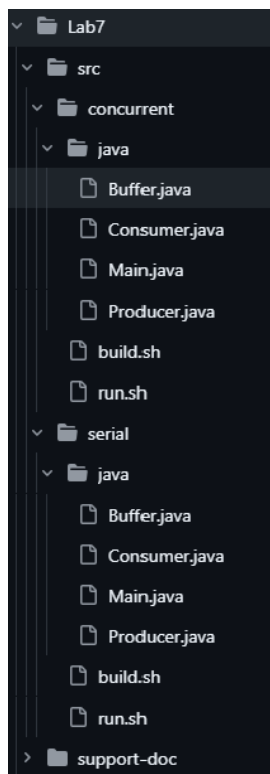
a) A solução deve funcionar corretamente com qualquer número de produtores e consumidores executando simultaneamente.

Prazo

16/06/2026 às 16h

Visão geral do código base

O código está organizado de acordo com a hierarquia abaixo:



- **src/serial/**: Diretório que disponibiliza a implementação serial do sistema produtor-consumidor.
- **src/concurrent/**: Diretório onde você implementará a versão concorrente do sistema produtor-consumidor. **Note que, inicialmente, este diretório contém uma cópia da implementação serial, então você deve alterá-la para implementar a concorrência!**

- **scripts auxiliares nos diretórios serial e concurrent:**
 - *build.sh*: script para compilar o código do diretório
 - *run.sh*: script para executar as implementações do diretório. Este exige os seguintes argumentos:
 - *<numero_produtores>*
 - *<numero_items_por_produtores>*
 - *<tempo_producao>* (em milisegundos)
 - *<numero_consumidores>*
 - *<tempo_consumo>* (em milisegundos)
- **support-doc**: Disponibiliza documentação e tutorial sobre o uso de threads, semáforos e sincronizações em Java.

Execução do código

1. Clone o repositório do código base

```
git clone [link do repositório]
```

2. Execução da versão serial do código

- a. Navegue até o diretório da implementação serial:

```
cd Lab7/src/serial
```

- b. Compile o código

```
bash build.sh
```

- c. Execute a versão serial especificando os parâmetros:

```
bash run.sh 1 100 150 1 150
```

3. Execução da versão concorrente do código

- a. Navegue até o diretório da implementação serial:

```
cd Lab7/src/concurrent
```

- b. Compile o código

```
bash build.sh
```

- c. Execute a versão serial especificando os parâmetros:

```
bash run.sh 1 100 150 1 150
```

Experimente diferentes configurações para ambiente de execução do sistema, por exemplo, com múltiplos produtores e consumidores e diferentes tempo de produção e consumo dos itens. Observe se o comportamento do sistema faz sentido em cada um dos cenários experimentados. Crie o arquivo **comments1.txt** dentro do diretório `src` e faça uma análise das execuções que você realizou. Comente o comportamento do sistema ao usar múltiplos produtores e consumidores com os diferentes tempos de produção e consumo dos itens.

Sugestão

Ao implementar a solução deste laboratório, sugerimos que você inicialmente desenvolva uma solução produtor-consumidor concorrente, e, em seguida, evolua esta solução para produtor-consumidor condicional. O desenvolvimento incremental da solução parece uma boa ideia.

Entrega

A entrega se dará através das atualizações do código no próprio repositório da dupla no Github Classroom.